

TITLE: EXP NO.: 3 IMPLEMENTING ASSET MANAGEMENT WITH HYPERLEDGER FABRIC

1. AIM

To implement an Asset Management application using Hyperledger Fabric by creating, reading, updating, transferring, and deleting assets through JavaScript chaincode and verifying the transactions on the blockchain ledger.

2. TOOLS / SOFTWARE REQUIRED

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

3. HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

4. PROCEDURE

Step 1: Navigate to the Fabric Test Network

Open the terminal and navigate to the Hyperledger Fabric test network.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

Start the Fabric network and create the application channel.

```
./network.sh up createChannel -ca
```

Step 3: Deploy the Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer chaincode.

```
./network.sh deployCC  
-ccl javascript  
-ccn basic  
-ccp ../asset-transfer-basic/chaincode-javascript
```

Step 4: Initialize the Ledger

Populate the ledger with sample assets.

```
peer chaincode invoke  
-o localhost:7050  
--ordererTLSHostnameOverride orderer.example.com  
--tls  
-cafile $ORDERER_CA  
-C mychannel  
-n basic  
-c '{"function":"InitLedger","Args":[]}'
```

Step 5: Query All Assets

Retrieve all asset records stored in the blockchain.

```
peer chaincode query  
-C mychannel  
-n basic  
-c '{"Args":["GetAllAssets"]}'
```

Step 6: Create a New Asset

Create a new asset named asset11.

```
peer chaincode invoke  
-o localhost:7050  
--ordererTLSHostnameOverride orderer.example.com  
--tls  
-cafile $ORDERER_CA  
-C mychannel  
-n basic  
-c '{"function":"CreateAsset","Args":["asset11","White","12","Arun","900"]}'
```

Step 7: Read the Asset Details

Retrieve the information of asset11.

```
peer chaincode query  
-C mychannel  
-n basic  
-c '{"Args":["ReadAsset","asset11"]}'
```

Step 8: Update Asset Information

Modify the asset information.

```
peer chaincode invoke
-o localhost:7050
--ordererTLSHostnameOverride orderer.example.com
--tls
--cafile $ORDERER_CA
-C mychannel
-n basic
-c '{"function":"UpdateAsset","Args":["asset11","Black","15","Arun","1200"]}'
```

Step 9: Transfer Asset Ownership

Transfer ownership of the asset to another user.

```
peer chaincode invoke
-o localhost:7050
--ordererTLSHostnameOverride orderer.example.com
--tls
--cafile $ORDERER_CA
-C mychannel
-n basic
-c '{"function":"TransferAsset","Args":["asset11","David"]}'
```

Step 10: Verify the Updated Asset

Query the asset again to verify the updated owner.

```
peer chaincode query
-C mychannel
-n basic
-c '{"Args":["ReadAsset","asset11"]}'
```

Step 11: Delete the Asset

Remove the asset from the ledger.

```
peer chaincode invoke
-o localhost:7050
--ordererTLSHostnameOverride orderer.example.com
--tls
--cafile $ORDERER_CA
-C mychannel
-n basic
-c '{"function":"DeleteAsset","Args":["asset11"]}'
```

Step 12: Verify Asset Deletion

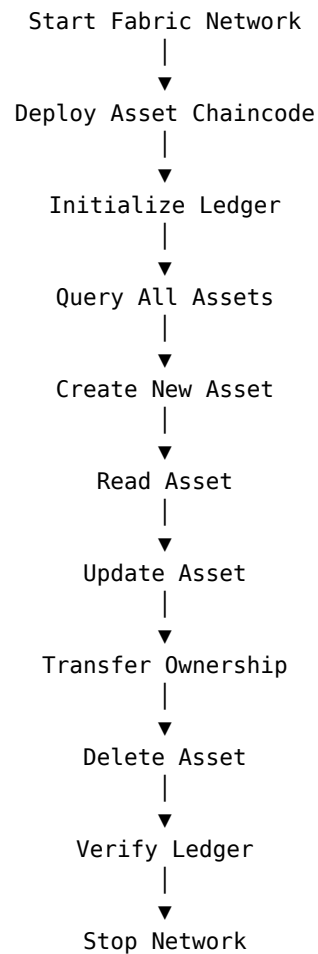
Attempt to read the deleted asset.

```
peer chaincode query
-C mychannel
-n basic
-c '{"Args":["ReadAsset","asset11"]}'
```

Step 13: Stop the Network

```
./network.sh down
```

5. FLOW OF PROCESS



6. SUMMARY OF KEY COMMANDS

Purpose	Command
Navigate	<code>cd ~/fabric-dev/fabric-samples/test-network</code>
Start network	<code>./network.sh up createChannel -ca</code>
Deploy chaincode	<code>./network.sh deployCC</code>
Invoke transaction	<code>peer chaincode invoke</code>
Query ledger	<code>peer chaincode query</code>
Stop network	<code>./network.sh down</code>

7. OUTPUT

The terminal output below is rendered from the output blocks supplied in the source experiment.

Output 1

A terminal window with a dark background and a title bar that reads "Experiment 3 - Fabric Output". The terminal displays a series of green text messages indicating successful operations in a Fabric network.

```
$ Fabric Network Started Successfully
Asset Chaincode Installed
Ledger Initialized
Assets Retrieved Successfully
New Asset Created
Asset Updated Successfully
Ownership Transferred Successfully
Asset Deleted Successfully
Ledger Verified Successfully
```

Fig. 3.1 – Fabric output corresponding to the source experiment output.

8. RESULT

The Asset Management application was successfully implemented using Hyperledger Fabric. The asset lifecycle operations, including asset creation, querying, updating, ownership transfer, and deletion, were executed successfully. The transactions were recorded on the blockchain ledger, demonstrating secure and immutable asset management in a permissioned blockchain network.